SynapseMemory

Autonomous Multi-Session Agentic Memory Platform powered by MongoDB Atlas

Event: MongoDB Student Hackathon SVNIT | **Track:** Track 1: Agentic Memory (PS 1) | **Branch:** vansh

1. The Problem Statement Explained in Plain English

The Everyday Amnesia Problem: Modern generative AI models (ChatGPT, Claude) possess no persistent memory across sessions. If an engineering team informs the AI: *'Our company strictly enforces the official MongoDB native driver, never Mongoose, with a connection pool size of 50,'* the AI respects it in that single chat. But when a teammate opens a new session tomorrow, the AI forgets everything and suggests non-compliant code. Teams waste hours repeating policies and fixing regressions.

The Hackathon Objective: Track 1 (Agentic Memory) specifically challenged participants to build an agent that remembers useful information, alters its behavior over time, and demonstrates persistence across interactions and sessions. Key requirements include deciding what to store, retrieving memories via **MongoDB Vector Search**, auto-expiring temporary tokens via **TTL Indexes**, resolving contradictions (conflict resolution), and broadcasting live database mutations via **Change Streams**.

2. What We Built & What Problem It Solves

The Solution: SynapseMemory. We built an enterprise-grade autonomous developer and SRE copilot backed by MongoDB Atlas. It acts as the *'Shared Cognitive Brain'* for engineering teams and AI agents. It captures architectural decisions, security boundaries, and outage post-mortems as rich documents. Whenever any engineer or autonomous bot writes code, SynapseMemory queries MongoDB using Vector Search, injects active constraints, detects contradictions, and guarantees compliance without manual human reminders.

Key Differentiators: *Beyond a basic chatbot, SynapseMemory features* ***3 Autonomous Collaborating Agents*** *(Architect, Security, SRE), an* ***Interactive 2D Force-Directed Knowledge Graph****, an* ***Ebbinghaus Memory Decay Engine****, an* ***Agentic PR Code Review Guard****, and an* ***SRE Outage Self-Healing Simulator****.*

3. Concrete System Outputs (What You See on Screen)

Output Component	Where It Appears	What It Conveys
MongoDB Memory Bank	Right Panel (Cards / Graph)	Displays persistent documents with status (active/superseded), recall count, and live TTL countdown.
Memory-Guided Chat	Left Panel (Interactive Copilot)	AI response citing exact memories: 'Applied Active Memories: [Strict Native Driver Policy]' and compliant code.
Contradiction Alert	Chat Feed Alert Card	Flags semantic conflict when a rule changes; marks old rule SUPERSEDED and activates the new rule.
Explainability Inspector	Modal Popup ('[Inspect]')	Displays cosine similarity (e.g. 92%), metadata filter match, and raw MongoDB JSON document.
Knowledge Graph	Memory Bank ('[Graph]' Tab)	Interactive 2D physics canvas visualizing relationships (enforces, superseded_by, indexes).
Multi-Agent Consensus	Multi-Agent Room Tab	Transcript of 3 agents cross-negotiating with a unified plan and Consensus Score (e.g. 96%).
PR Review Audit	PR Code Guard Tab	Annotated code lines flagging forbidden ORMs or pool starvation risks with auto-generated fixes.
SRE Self-Healing	SRE Self-Healing Tab	Timeline showing latency drop (4,320ms -> 12ms) via INC-402 runbook and logging a new memory.

4. Technical Pipeline Architecture & Workflow

SynapseMemory connects Next.js 14, vector embedding models, and MongoDB Atlas into an event-driven architecture:

- **1. Extraction & Ingestion:** Analyzes developer queries, architectural statements, or post-mortems. Generates 768-dimensional normalized vector embeddings and writes to the memories collection.
- **2. Semantic Vector Recall ($vectorSearch):** Recalls relevant memories using cosine vector distance with strict metadata pre-filtering on workspaceId and lifecycle.status: 'active'.
- **3. Conflict Detection Engine:** When a new constraint arrives, the engine semantically evaluates active memories. If an explicit contradiction is detected, the preceding rule is automatically updated to status: 'superseded' with an audit trail.
- **4. TTL Auto-Expiry Engine:** Employs a MongoDB TTL index ({ expiresAt: 1 } with expireAfterSeconds: 0) to automatically purge ephemeral credentials and incident war-room tokens.
- **5. Change Streams (watch()):** Server-Sent Events (SSE) listener watches database mutations and broadcasts live updates directly to connected web clients in real time.
- **6. Ebbinghaus Memory Decay Pipeline:** Advanced aggregation pipeline calculating exponential retention curves: Retention = exp(-Δt / (τ * (1 + ln(1 + N)))).

5. Step-by-Step Testing & Judging Walkthrough Guide

Open **http://localhost:3000** in any browser and execute these 9 steps in sequence to evaluate all project capabilities:

Step & Action	Trigger	Observed Result	Judging Rubric Proven
Step 1: Load Demo Scenarios	Click green 'Load Demo Scenarios' button in header.	Memory Bank populates with 3 baseline documents: (1) Strict MongoDB Native Driver Policy, (2) Incident INC-402 Fix, (3) War-Room Token with 120s TTL.	Proves rich document modeling with lifecycle and TTL fields.
Step 2: Session Continuity Test	Switch dropdown to 'Session 2' and click '2. Later Session Recall Test'.	Assistant responds with official native driver code, pool size 50, and TLS 1.3, citing both recalled memories without reminder.	Proves memory persists across sessions and visibly alters behavior.
Step 3: Explainability Inspector	Click the eye icon ('[Inspect]') on any memory card or graph node.	Modal opens showing Cosine Similarity Score (e.g. 92%), filter match checks, and raw MongoDB JSON document.	Fulfills Step 3: Inspect memory and explain selection.
Step 4: Contradiction Resolution	Click '3. Conflict & Contradiction Test' (migrating to Prisma ORM).	Amber alert confirms contradiction. Old native driver policy turns red (SUPERSEDED); new Prisma rule becomes ACTIVE.	Proves handling difficult cases (conflict/correction/superseding).
Step 5: Ephemeral TTL Countdown	Observe purple 'Temporary War-Room Token' card in Memory Bank.	Live timer counts down in seconds until MongoDB TTL index automatically purges the ephemeral document.	Proves automated time-based memory expiry.
Step 6: Knowledge Graph Canvas	Click the Graph icon ('[Graph]') in Memory Bank header.	60fps interactive 2D physics canvas renders memory nodes. Drag nodes around and click to inspect.	Visualizes cognitive knowledge map of team memory.
Step 7: Multi-Agent Consensus	Click 'Multi-Agent Room' tab and click 'Run Deliberation'.	3 agents (Archi, Aegis, Sentinel) cross-negotiate and output a harmonized plan with Consensus Score: 96%.	Fulfills brief's 'Shared Context for Teammates' Agents' project.
Step 8: PR Code Review Guard	Click 'PR Code Guard' tab and run review on PR #104.	Flags forbidden Mongoose import and pool starvation risk with inline line annotations; auto-generates fix.	Demonstrates CI/CD memory enforcement on code diffs.
Step 9: SRE Incident Self-Healing	Click 'SRE Self-Healing' tab and trigger autonomous healing.	Recalls INC-402, dynamically rebalances pool, drops latency (4,320ms -> 12ms), and logs new episodic memory.	Demonstrates adaptive incident response loop.

6. Strict Hackathon Rubric Compliance Matrix

Direct point-by-point verification demonstrating that SynapseMemory fulfills 100% of the requirements from MongoDB Student Hackathon SVNIT.pdf:

Hackathon Requirement (from PDF)	Implementation in SynapseMemory	Compliance Status
Document Modeling: Model each memory as a document with owner, type, content, source, timestamps, lifecycle fields.	MongoDB collection memories stores rich schema with userId, workspaceId, type, source, createdAt, and lifecycle (status, confidence, accessCount, supersededBy).	**100% COMPLIANT**
Vector Search: Use MongoDB Vector Search for semantic recall with metadata filters.	Implemented via $vectorSearch pipeline using 768-dim embeddings with metadata pre-filtering on workspaceId and lifecycle.status: 'active'.	**100% COMPLIANT**
TTL Indexes: Use a TTL index on a date field for time-based memory expiry.	Compound TTL index { expiresAt: 1 } with expireAfterSeconds: 0 automatically purges expired memories at the database engine level.	**100% COMPLIANT**
Change Streams: Use change streams when multiple users or agents need live updates.	MongoDB collection.watch() piped through Server-Sent Events (SSE) route /api/memories/stream dynamically updates browser clients in real time.	**100% COMPLIANT**
Demo Flow 1: Start with an interaction that creates a useful memory.	Ingests architectural policies, security constraints, and incident post-mortems via interactive chat or one-click demo loaders.	**100% COMPLIANT**
Demo Flow 2: Open a later session and show retrieved memory changing result or action.	In Session 2, the agent retrieves stored constraints and generates code adhering strictly to native driver and pool size rules without reminders.	**100% COMPLIANT**
Demo Flow 3: Inspect stored memory and explain why it was selected.	The Explainability Inspector modal displays cosine similarity scores, filter match checks, and raw MongoDB JSON documents.	**100% COMPLIANT**
Demo Flow 4: Demonstrate one difficult case: a correction, expiry, deletion, conflict.	Demonstrates semantic contradiction detection (old rule marked SUPERSEDED; new rule activated) and TTL time-based auto-expiry.	**100% COMPLIANT**
Suggested Idea: Shared Context for Teammates' Agents.	Implemented in Multi-Agent Room where 3 autonomous agents (Archi, Aegis, Sentinel) negotiate on shared MongoDB memory.	**100% COMPLIANT**

7. Repository Status & Submission Verification

- **Git Branch:** vansh (All commits clean, zero untracked files).
- **Production Build:** npm run build passed with 0 errors across all 8 dynamic routes.
- **Containerization:** Production Dockerfile and docker-compose.yml ready.
- **Live Local Server:** Running at http://localhost:3000 ready for demonstration.